

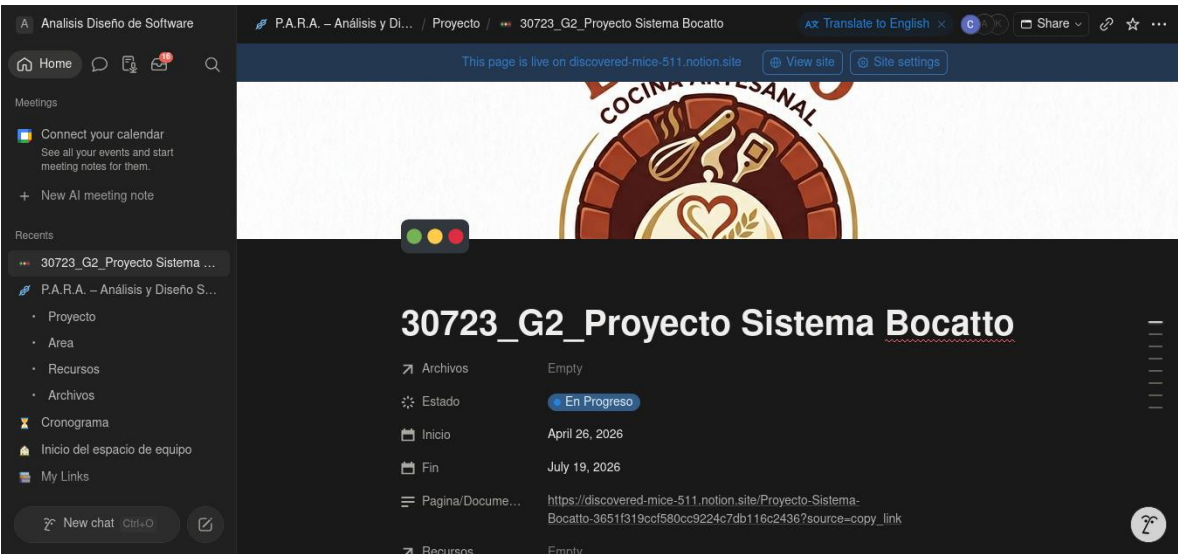
A.- OBJETIVO.-

Desarrollar un sistema web de gestión de pedidos para el restaurante Bocatto Valley que cubra el ciclo completo de atención (toma del pedido, preparación en cocina y cobro en caja), soportando los tres canales de venta del negocio (Mesa, QR y Telefónico), con cinco roles de usuario diferenciados, y aplicando la metodología SCRUM.

S	Específico	Administrar el flujo integral de pedidos, incluyendo registros en mesa, telefónicos y mediante códigos QR, además de la gestión en cocina y cobro en caja.
M	Medible	Cumplimiento y validación de 8 requisitos funcionales específicos (RF-01 al RF-08) y sus interfaces correspondientes.
A	Alcanzable	Desarrollo bajo la metodología AUP (Agile Unified Process), utilizando tecnologías como React para el frontend y Node.js o Java para el backend, con bases de datos relacionales.
R	Relevante	Optimizar la atención al cliente y garantizar el cumplimiento de normativas legales (ARCSA y Protección de Datos Personales).
T	Temporal	Ciclo de revisión inicial completado en junio de 2026 .

B.- DESARROLLO.-

A través de todo el proyecto se utilizó notion para documentar y hacer el seguimiento de todas las actividades realizadas durante el desarrollo.



Elicitación de Requisitos

Este fue el punto de partida, el SRS detalla formalmente los requisitos que el sistema debe cumplir.

3.2 Requisitos funcionales

3.2.1 Autenticar en el sistema

[Autenticar en el sistema](#)

3.2.2 Tomar pedido en mesa

[Tomar pedido en mesa](#)

3.2.3 Registrar pedido en mesa

[Registrar pedido en mesa](#)

3.2.4 Comunicar pedido en cocina

[Comunicar pedido a cocina](#)

3.2.5 Tomar pedido telefónico

[Tomar pedido telefónico](#)

3.2.6 Registrar pedido telefónico

[Registrar pedido telefónico](#)

3.2.7 Gestionar pedido por QR

[Gestionar pedido por QR](#)

El siguiente paso fue realizar la matriz IREB, con el fin de comprobar los criterios de calidad de los requisitos

N15

fx

A

B

C

D

E

F

G

H

I

J

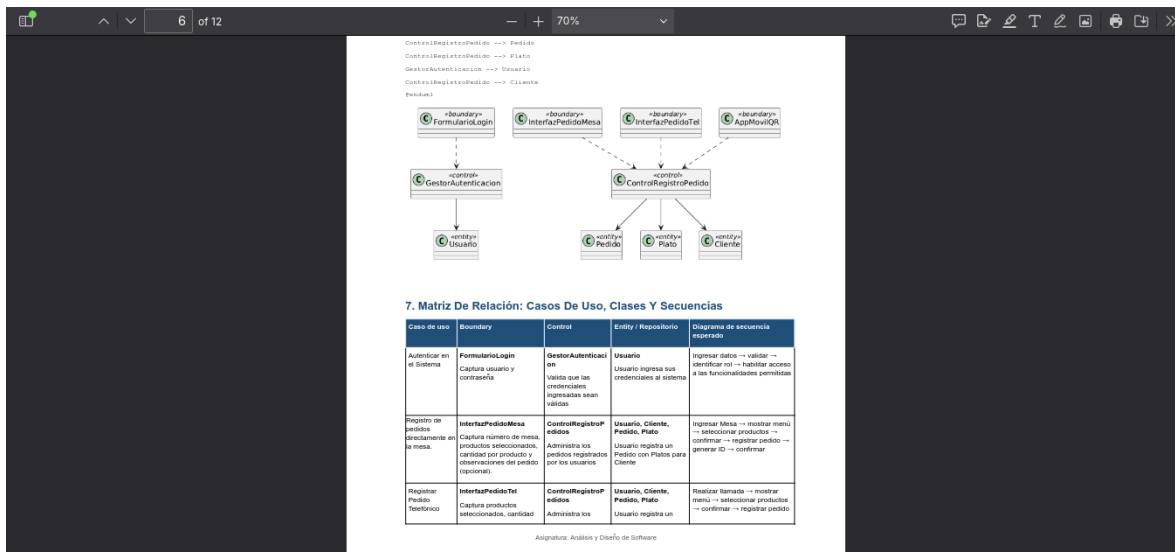
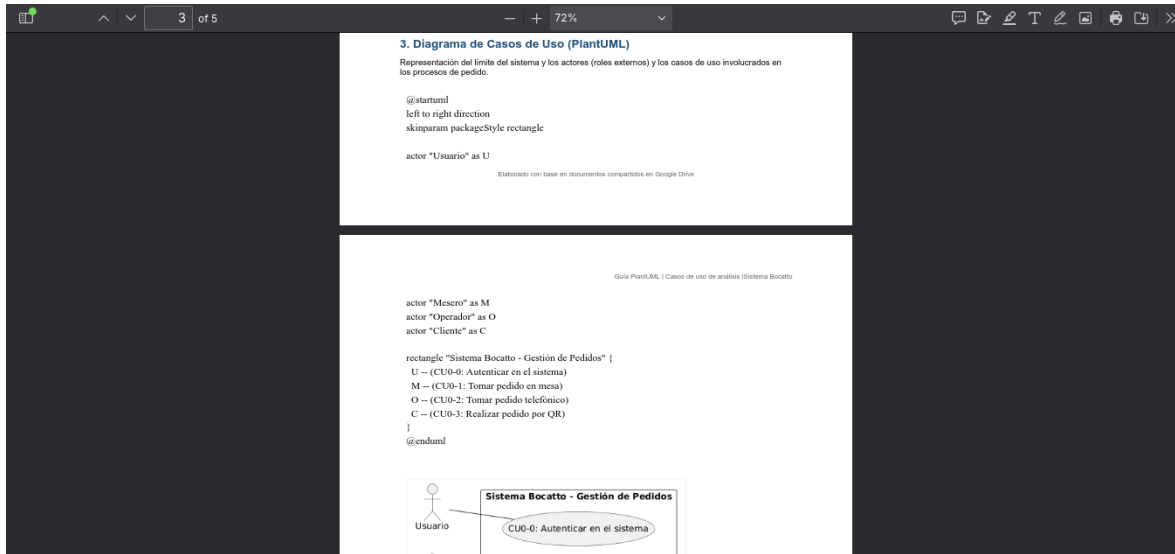
MATRIZ PARA COMPROBACIÓN REQUERIMIENTOS DE CALIDAD DE PROYECTO ACADEMICO

SISTEMA:

SISTEMA DE GESTION DE PEDIDOS (Bocatto Valley)

			Requerimientos Funcionales		
Nº	CRITERIOS DE CALIDAD (Documento/Requerimientos)	Status Nivel de Acuerdo	CU0-0 Autenticar en el sistema	CU0-1 Tomar pedido en mesa	CU0-2 Tomar pedido telefónico
1	Agreed	OK	1. Videoconferencia de la entrevista 2. Minuta de la entrevista 3. Aceptación por videoconferencia	OK	1. Videoconferencia de la entrevista 2. Minuta de la entrevista 3. Aceptación por videoconferencia
2	Ranked	OK	1. En la descripción del caso de Uso se especifica la prioridad Alta y es de tipo Soporte	OK	1. En la descripción del caso de Uso se especifica la prioridad Alta y los tipos Primario y Esencial
3	Unambiguos	OK	1. Redacción en lenguaje claro. 2. El tutor y el equipo interpretan de igual manera el caso de uso	OK	1. Redacción en lenguaje claro. 2. El tutor y el equipo interpretan de igual manera el caso de uso
4	Valid and up-to date	NOK	1. No se tuvo un feedback final por parte del cliente. 2. Está alineado con los objetivos del sistema.	NOK	1. No se tuvo un feedback final por parte del cliente. 2. Está alineado con los objetivos del sistema.
5	Correct	OK	1. Refleja lo que realmente necesita el	OK	1. Refleja lo que realmente necesita el

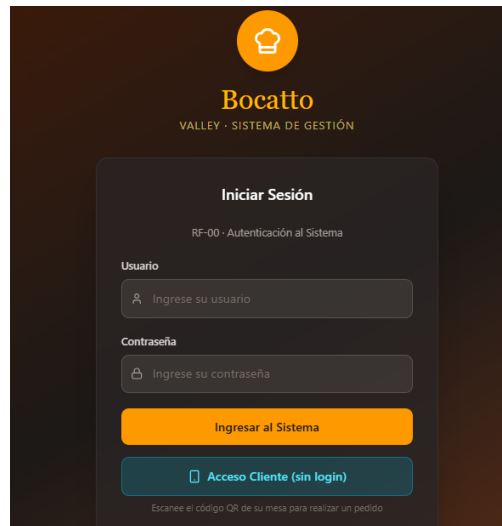
Luego de eso, se llevó a cabo la realización de 2 guías, la primera para “Desarrollar en plantuml diagramas de casos de uso de análisis” y la segunda para identificar la “Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis”. Ambas utilizando plantuml.



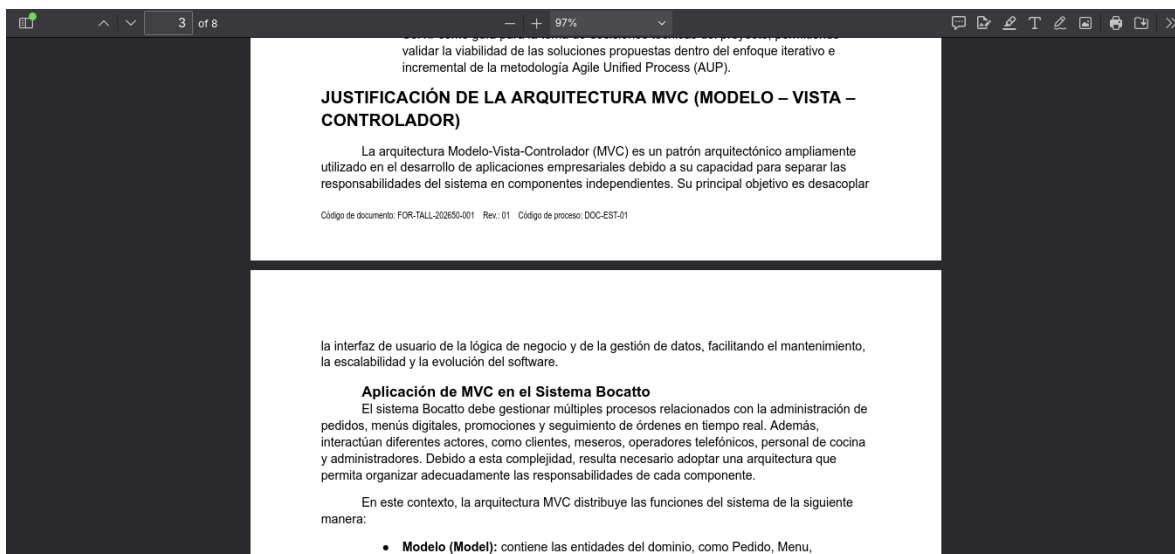
También se crearon dos cuadernos de notebooklm, uno de diseño y otro de análisis, y un prototipo

<https://notebooklm.google.com/notebook/d8ea8d0a-a9c9-4ef7-b809-0b9819e63d4b>

<https://notebooklm.google.com/notebook/8531f7f2-9ea1-4d2d-8296-7749c3fafa2f>

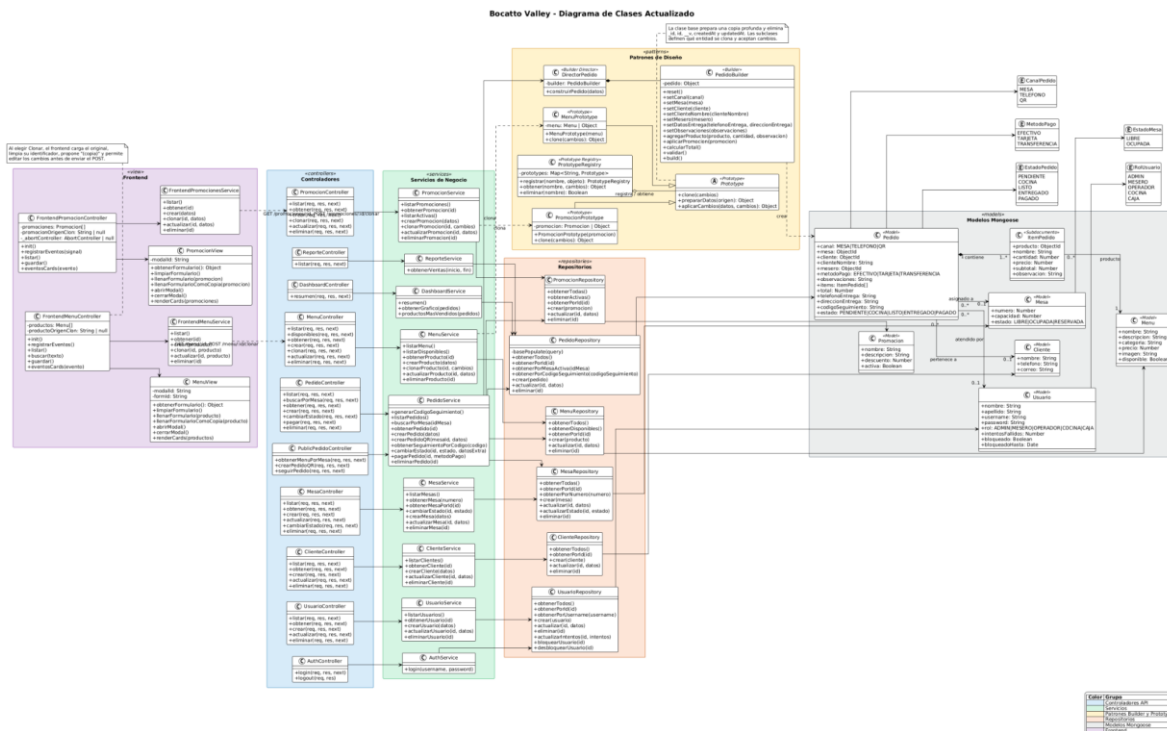
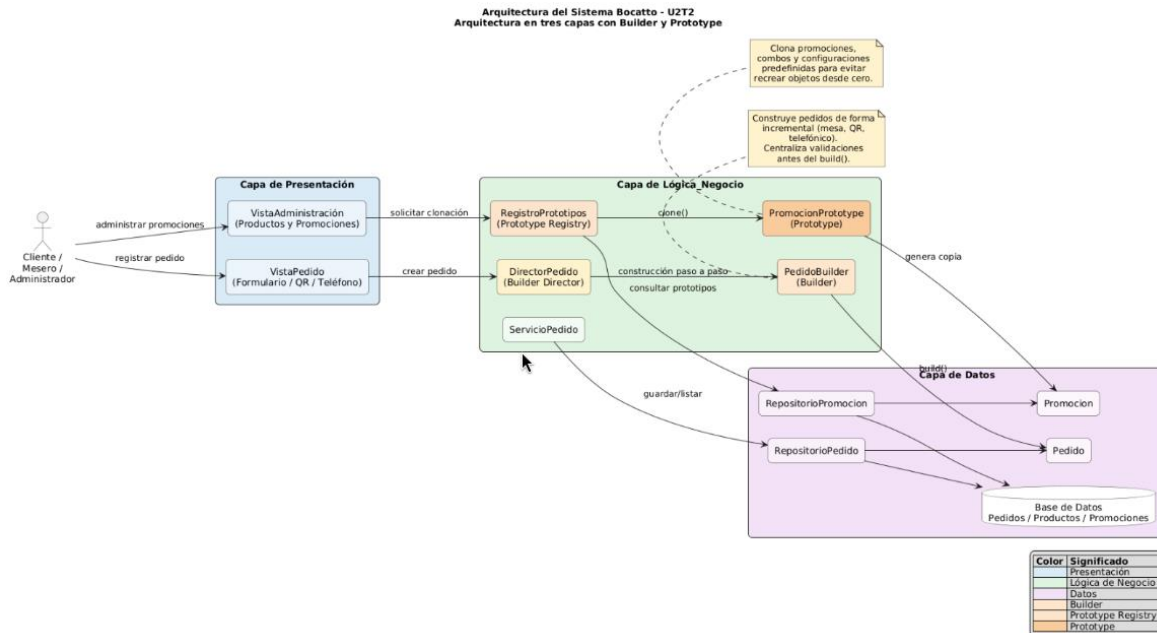


Se procedió con la selección de la arquitectura a utilizar, en este caso MVC. Además de la elección de dos patrones de diseño, Builder y Prototipe. Además se creó un cuaderno de Construcción en NotebookLM

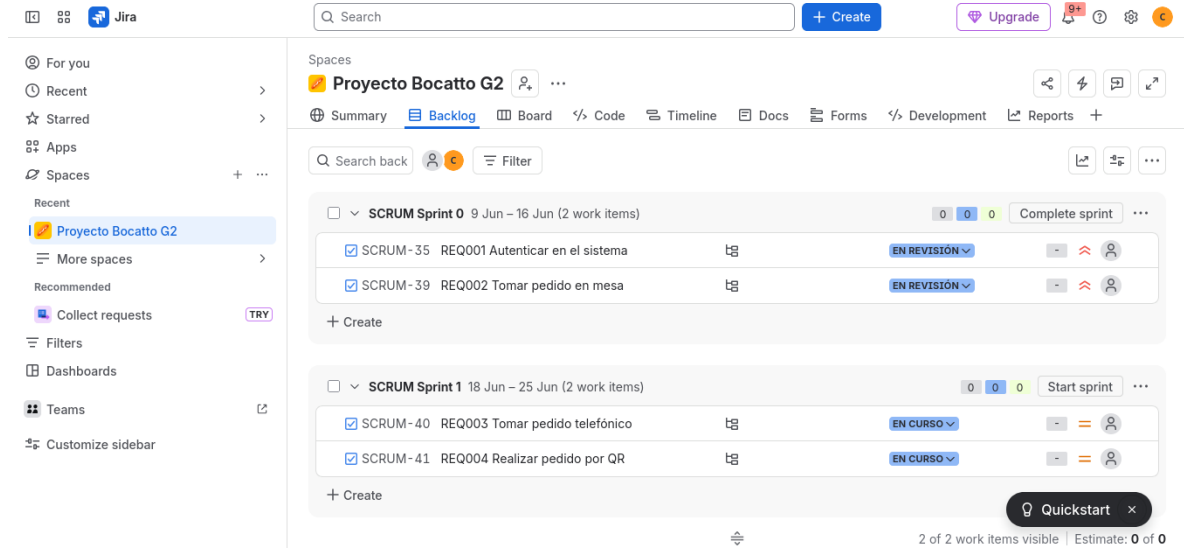


<https://notebooklm.google.com/notebook/07456efe-69d3-4d57-8c6b-a95e26f3355f>

También fue realizada la propuesta de la arquitectura en PlantUML, y los diagramas de clase a ser utilizados



Con todo esto se empezó a trabajar con jira, definiendo sprints para los requisitos junto con sus subtarear, y un prototipo incluido en este.



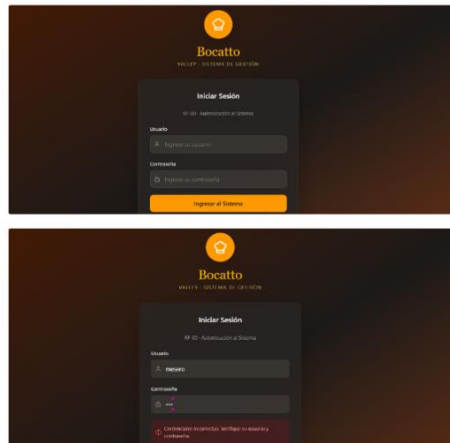
The screenshot shows the Jira interface for 'Proyecto Bocatto G2'. The left sidebar contains navigation options like 'For you', 'Recent', 'Starred', 'Apps', 'Spaces', 'Filters', 'Dashboards', 'Teams', and 'Customize sidebar'. The main area displays the 'Backlog' view with two sprints:

- SCRUM Sprint 0** (9 Jun – 16 Jun, 2 work items):
 - SCRUM-35 REQ001 Autenticar en el sistema (Status: EN REVISIÓN)
 - SCRUM-39 REQ002 Tomar pedido en mesa (Status: EN REVISIÓN)
- SCRUM Sprint 1** (18 Jun – 25 Jun, 2 work items):
 - SCRUM-40 REQ003 Tomar pedido telefónico (Status: EN CURSO)
 - SCRUM-41 REQ004 Realizar pedido por QR (Status: EN CURSO)

At the bottom right, it indicates '2 of 2 work items visible | Estimate: 0 of 0'.

REQ001 Validación de Credenciales

De KIRILLY ALEXANDRA CHORIBOGA CABRERA Escuchar 4 Añade una reacción



Después de esto se realizó la creación de un tablero kanban, que cumple una función similar a la de jira, ser una herramienta visual para la gestión del proyecto.

Estados	Criticidad	Tarea	Descripción	Inicio	Previsto	Final	Duración	Retraso	Asignado
COMPLETADO	IMPORTANTE	Sprint 0: REQ001-1	Construcción de interfaz FormularioLogin.	6/9/2026	6/11/2026	6/11/2026	3 días	0 días	Kerly Chiriboga
COMPLETADO	IMPORTANTE	Sprint 0: REQ001-2	Implementación de métodos y control de registro.	6/9/2026	6/11/2026	6/11/2026	3 días	0 días	Carlos Caiza
COMPLETADO	IMPORTANTE	Sprint 0: REQ001-3	Control de seguridad: Bloqueo de cues	6/9/2026	6/11/2026	6/11/2026	3 días	0 días	Alexander Villacre
COMPLETADO	IMPORTANTE	Sprint 0: REQ002-1	Diseño de interfazPedidoMesa.	6/12/2026	6/15/2026	6/15/2026	4 días	0 días	Kerly Chiriboga
COMPLETADO	IMPORTANTE	Sprint 0: REQ002-1	Implementar métodos tomarPedidoEi	6/12/2026	6/15/2026	6/15/2026	4 días	0 días	Carlos Caiza
COMPLETADO	IMPORTANTE	Sprint 0: REQ002-3	Lógica de validación: validarDatosDelP	6/12/2026	6/15/2026	6/15/2026	4 días	0 días	Alexander Villacre
COMPLETADO	IMPORTANTE	Sprint 1: REQ003-1	Interfaz InterfazPedidoTel con campos	6/18/2026	6/21/2026	6/21/2026	4 días	0 días	Carlos Caiza
COMPLETADO	IMPORTANTE	Sprint 1: REQ003-2	Método validarClienteExistente(tel) ps	6/18/2026	6/21/2026	6/21/2026	4 días	0 días	Kerly Chiriboga
COMPLETADO	IMPORTANTE	Sprint 1: REQ003-3	Integración final del RepositorioPedid	6/18/2026	6/21/2026	6/21/2026	4 días	0 días	Alexander Villacre
COMPLETADO	IMPORTANTE	Sprint 1: REQ004-1	Construcción de la interfaz de pedido i	6/22/2026	6/25/2026	6/25/2026	4 días	0 días	Alexander Villacre
COMPLETADO	IMPORTANTE	Sprint 1: REQ004-2	Implementación de la lógica leerCodig	6/22/2026	6/25/2026	6/25/2026	4 días	0 días	Carlos Caiza
COMPLETADO	IMPORTANTE	Sprint 1: REQ004-3	Desarrollo de la funcionalidad presentarMenuDigital() consultando los items de la entidad Menu.	6/22/2026	6/25/2026	6/25/2026	4 días	0 días	Alexander Villacre
EN ESPERA	IMPORTANTE	Sprint 1: REQ004-4	Programación del método realizarPedidoPorQR() en el objeto ControlRegistroPedido.	6/22/2026	6/25/2026	6/25/2026	4 días	0 días	Kerly Chiriboga
EN ESPERA	IMPORTANTE	Sprint 1: REQ004-5	Implementación del flujo de confirmac	6/22/2026	6/25/2026	6/25/2026	4 días	0 días	Carlos Caiza
EN PROGRESO	IMPORTANTE	Sprint 2: REQ005-1	Implementar la interfaz para el Cocine	6/27/2026	7/4/2026	7/4/2026	8 días	0 días	Kerly Chiriboga
EN PROGRESO	IMPORTANTE	Sprint 2: REQ005-2	Implementación de métodos en Contr	6/27/2026	7/4/2026	7/4/2026	8 días	0 días	Carlos Caiza

Este tablero ayudó a poder finalmente completar la parte funcional del proyecto.

Pero además de la parte funcional, se realizará un plan de pruebas para ser implementadas en el proyecto

4 of 44

Automatic Zoom

- Backend** — Node.js 20+, Express 5, Mongoose 8, MongoDB, autenticación con JWT, Socket.IO.
- Frontend** — SPA en JavaScript puro (sin framework de build), estructura modular (servicios, controladores, vistas, validadores, utilidades), plantillas HTML, CSS, cliente de Socket.IO.

2.3 Módulos del proyecto bajo prueba

#	Módulo	Archivos backend	Archivos frontend
1	Autenticación	services/AuthService.js, middlewares/authMiddleware.js, middlewares/roleMiddleware.js	services/authService.js, utils/Auth.js, utils/storage.js
2	Usuarios	services/UsuarioService.js, validators/UsuarioValidator.js, dto/UsuarioDTO.js, mappers/UsuarioMapper.js	services/usuarioService.js, validators/UsuarioValidator.js, views/UsuarioView.js
3	Clientes	services/ClienteService.js, validators/ClienteValidator.js	services/clienteService.js, validators/ClienteValidator.js
4	Mesas	services/MesaService.js, validators/MesaValidator.js, dto/MesaDTO.js, mappers/MesaMapper.js	services/mesaService.js, validators/MesaValidator.js
5	Menú	services/MenuService.js, validators/MenuValidator.js, patterns/prototype/MenuPrototype.js	services/menuService.js, validators/MenuValidator.js
6	Pedidos	services/PedidoService.js, validators/PedidoValidator.js, patterns/builder/PedidoBuilder.js, patterns/builder/DirectorPedido.js, dto/PedidoDTO.js	services/pedidoService.js, validators/PedidoValidator.js, utils/EstadoPedido.js
7	Cocina	services/CocinaService.js	services/cocinaService.js, views/CocinaView.js
8	Caja	services/CajaService.js	services/cajaService.js, views/CajaView.js
9	Promociones	services/PromocionService.js, validators/PromocionValidator.js, patterns/prototype/PromocionPrototype.js	services/promocionesService.js, views/PromocioneView.js
1	Report	services/ReportService.js	services/reporteService.js

Luego de su implementación, se llegó a su ejecución y se obtuvo el 100% de éxito en ambos, el frontend y el backend

```
.../3.Codigo/BOCATTO_V3.0/frontend

✓ assets/js/qr-order-helpers.spec.js (20)
✓ services/mesaService.spec.js (7)
✓ services/usuarioService.spec.js (6)
✓ services/promocionesService.spec.js (6)
✓ services/menuService.spec.js (6)
✓ services/clienteService.spec.js (6)
✓ services/pedidoService.spec.js (5)
✓ services/cajaService.spec.js (5)
✓ utils/Auth.spec.js (9)
✓ utils/storage.spec.js (7)
✓ services/cocinaService.spec.js (3)
✓ validators/PedidoValidator.spec.js (7)
✓ services/authService.spec.js (2)
✓ utils/Toast.spec.js (5)
✓ validators/MesaValidator.spec.js (7)
✓ services/reporteService.spec.js (2)
✓ validators/UsuarioValidator.spec.js (6)
✓ services/dashboardService.spec.js (2)
✓ validators/MenuValidator.spec.js (5)
✓ utils/EventBus.spec.js (3)
✓ utils/Modal.spec.js (4)
✓ validators/ClienteValidator.spec.js (3)
✓ utils/EstadoPedido.spec.js (1)
✓ services/api.spec.js (1)

Test Files 24 passed (24)
  Tests 128 passed (128)
Start at 23:47:26
Duration 4.29s (transform 262ms, setup 2ms, collect 641ms, tests 262ms, environment 16.53s, prepare 3.95s)

[eluce@eris] > ~/Code/Analisis/KerllyChiriboga_30723_G2_ADSW/3. Codigo/BOCATTO_V3.0/frontend <main*>
>>>
```

```
.../3. Codigo/BOCATTO_V3.0/backend

✓ rechaza nombre vacío (2 ms)
✓ rechaza descuento fuera de rango (BVA: -1, 101) (2 ms)

PASS src/services/ClienteService.spec.js
  ClienteService
    ✓ listarClientes delega en obtenerTodos (3 ms)
    ✓ obtenerCliente delega en obtenerPorId
    ✓ crearCliente delega en crear (1 ms)
    ✓ actualizarCliente delega en actualizar
    ✓ eliminarCliente delega en eliminar (1 ms)

PASS src/validators/MesaValidator.spec.js
  MesaValidator
    ✓ acepta una mesa válida (2 ms)
    ✓ rechaza numero faltante (3 ms)
    ✓ rechaza numero falsy (0) (1 ms)
    ✓ rechaza capacidad < 1 (BVA: 0 y negativo) (1 ms)

PASS src/mappers/MesaMapper.spec.js
  MesaMapper
    ✓ toDTO envuelve una sola entidad (3 ms)
    ✓ toDTOList envuelve un arreglo de entidades (1 ms)
    ✓ toDTOList maneja un arreglo vacío (1 ms)

Test Suites: 24 passed, 24 total
Tests: 154 passed, 154 total
Snapshots: 0 total
Time: 1.057 s, estimated 2 s
Ran all test suites.

[eluce@eris] > ~/Code/Analisis/KerllyChiriboga_30723_G2_ADSW/3. Codigo/BOCATTO_V3.0/backend <main*>
>>> |
```

C.- CONCLUSIONES.-

Bocatto V3.0 implementa el ciclo completo de gestión de pedidos del restaurante (toma, cocina, caja) en los tres canales de venta (Mesa, QR, Telefónico), con cinco roles RBAC diferenciados (ADMIN, MESERO, OPERADOR, COCINA, CAJA), cumpliendo el objetivo general planteado en la sección A.

Aunque no se mencionó durante el desarrollo, el notion estuvo actualizándose constantemente para reflejar el estado actual del proyecto, añadiéndole información y actualizando cronogramas, este además fue utilizado como el reemplazo de jira una vez se terminó el periodo de prueba de este último. Las pruebas realizadas ayudaron a verificar que la parte funcional del proyecto esté correcta, tanto en el frontend como en el backend, e plan de pruebas guió este proceso, ayudando a identificar qué pruebas le pertenecen a cada requisito funcional y qué elementos estarán involucrados.

D.- RECOMENDACIONES.-

Actualizar constantemente el notion mientras se realiza el proyecto, ya que elementos importantes pueden ser omitidos si se deja para más tarde.

Añadir una sección en el notion o en el repositorio de git con imágenes acerca de cómo fue evolucionando el proyecto, para que estos cambios se puedan observar de forma visual y sea más fácil entender qué ha cambiado.

Investigar acerca de los frameworks de prueba disponibles para determinar cual es el más apropiado para ser utilizado en el proyecto, ya sea por su fácil integración, completitud o facilidad de uso.

Sangolquí, 23 de julio de 2026

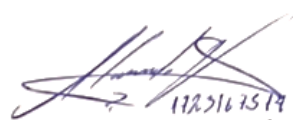
Elaborado Por:



Caiza Carlos
ingeniería de Software,
presencial, Matriz
Sangolquí



Kerly Chiriboga
ingeniería de Software,
presencial, Matriz
Sangolquí



Alexander Villacrés
ingeniería de Software,
presencial, Matriz
Sangolquí